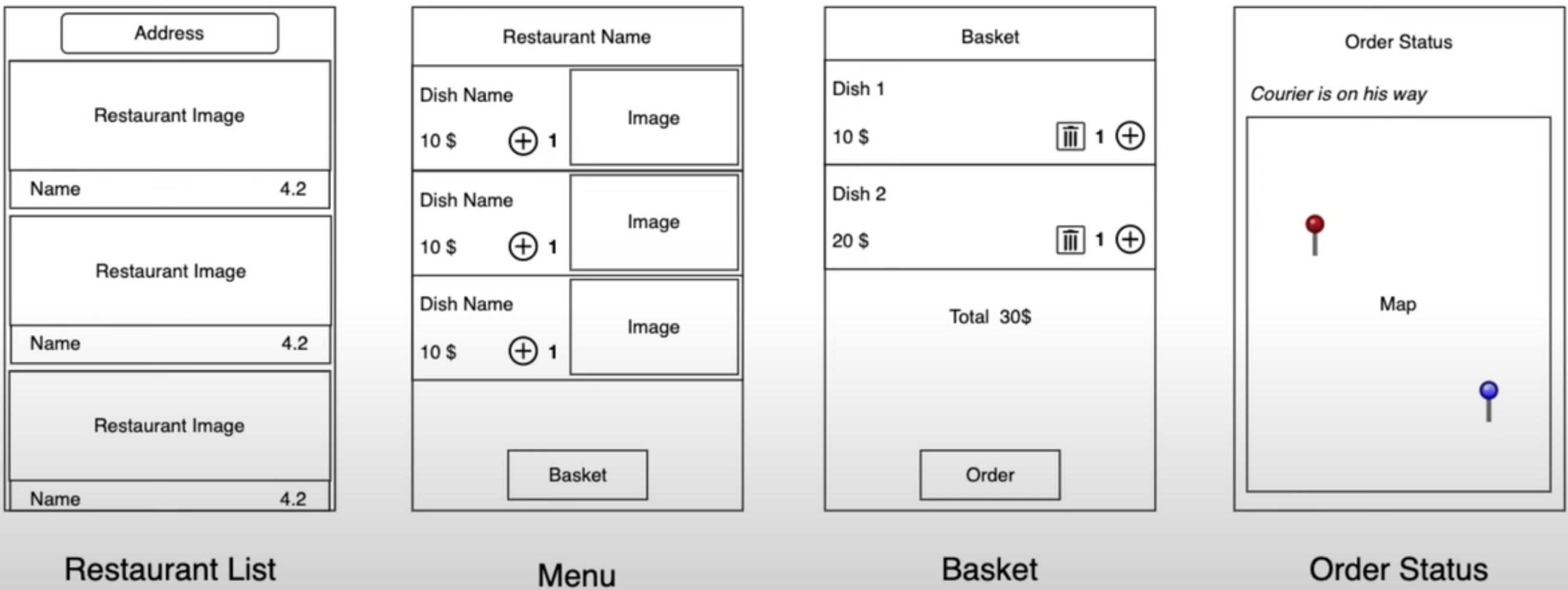


Problem description

Design a simple version of
the Uber Eats App



Data Model Design

Functional Requirements:

- Support logical "addresses", current address and delivery address. Based on location but user can override.
- Display all recommended merchants near "current" address in a list-view.
- If a user clicks on a merchant, display the merchant's menu/catalogue in a Menu list-view
- User's should be able to create a shopping cart/basket invoice by adding items
- User's can place an order by checking out a basket.
- Ignore payment processing, if a user taps checkout or order, assume its placed.
- A map view should display a current order's status in real-time

Non-Functional Requirements:

- Optimize for fast, smooth, responsive UX, especially the list-views.
- Optimize for network latency and reduce client-server trips
- Application should work in poor networks
- Complete offline support is not needed (not expected to work when there is no connectivity). This app depends on some baseline network connectivity.
- Don't assume unbounded disk space (can cache for better performance but assume limited to moderate disk space).
- Optimize for battery usage (reduce battery usage).

Localization & Translations

- All strings sent by the server will be localized and translated. So similar to what we did at Facebook the server side will handle localization and translations for us.
- When sending a request, we will send the current device's locale along with request, modern HTTPS have a field called "Accepted Language" in the header.
- Serverside will then use the locale to pick the return the proper translated string.
- Any string on the client-side only will use the standard iOS Infra for localization (Localizable.string) bundle files.

Currency Localization

Currency localization involves formatting the display of currency to respect the user's currencyCode (locale currency). It also involves calculating invoices (adding prices) correctly based on the currency code. Some currencies use different precision, different rounding rules, and apply taxes differently which can all affect how the currency is localized.

- Approach taken is to have server return a currency neutral "Price", that simply has amount as a integer, num decimal places, and price's currencyCode. Its then up to the client to take this information and use native number formatters to format the display of the 'price' to respect the device's locale.
- Took this approach because my design calculates invoices on the client side itself, so since the client is handling all the complex locale specific currency calculation logic, made sense to offload the display localization of currencies to the client as well.
- I believe this turned out to be the *WRONG* decision, please read the "Data Model Design & Feedback" section for more.

```
Address {
  location: CLLocation
  street: String
  city: String
  postalCode: String
  buildingNo: String?
  apartmentNo: String?
}
```

```
Merchant {
  merchantID: String
  name: String
  imageURI: URL
  starRating: Float
  reviewCount: Int
  address: Address
  distance: Float
  estimatedDeliveryTime: Int
  description: String?
}
```

```
/*
 * I did not flesh these out to
 * prevent data modeling scope creep
 * but its your standard User object.
 * Its possible Courier is a "User",
 * or its possible its a standalone object.
 */
User {
  userID: String
  email: String
  phone: String
  firstName: String
  lastName: String
  addresses: [Address]
}
```

```
Courier {}
```

```
Price {
  amount: Int,
  currencyCode: String,
  places: Int
}
```

```
ProductCatalogue {
  merchantID: String
  catalogueID: String
  description: String?
  items: [ProductCategory : ProductCatalogueItem]
}
```

```
ProductCategory: Hashable {
  category: String // This is a unique key
  description: String?
}
```

```
ProductCatalogueItem {
  itemID: String
  catalogueID: String
  price: Price
  name: String
  imageURI: URL?
  description: String?
}
```

```
ShoppingCartItem : ProductCatalogueItem {}
```

```
ShoppingCartDisplayTotal {
  total: String
  count: Int
  name: String
}
```

```
ShoppingCartBasket {
  basketID: String //Created on client
  merchantID: String
  userID: String
  cartInvoice: ShoppingCartInvoice
  displayTotals: [ShoppingCartDisplayTotal]
```

```
API:
  addItem(ShoppingCartItem)
  getCartTotals() -> [ShoppingCartDisplayTotal]
  getInvoice() -> Invoice
}
```

```
// Creates Invoice Items from ShoppingCartItem's
InvoiceItemFactory {}
```

```
InvoiceItem {
  itemID: String
  name: String
  price: Price
  discount: Float?
  tax: Float?
  .....
  ....
}
```

```
InvoiceItemTotal {
  invoiceItem: InvoiceItem
  adjustedTotal: Float
  adjustedTotalLocalized: String
}
```

```
Invoice {
  invoiceID: String
  invoiceItems: [[InvoiceItemTotal]]
  tax: Float?
  discount: Float?
  subTotal: Float
  subTotalLocalized: String
  total: Float
  totalLocalized: String
}
```

```
API:
  addInvoiceItem()
  addInvoiceItemForKey()
  getInvoiceItemTotals()
}
```

```
OrderStatus (Enum) {
  OrderPlaced
  MerchantRecieved
  MerchantConfirmed
  CourierDispatched
  CourierEnRoute
  OrderDelivered
}
```

```
Order {
  orderID: String
  invoice: Invoice
  status: OrderStatus
  merchant: Merchant
  deliveryAddress: Address
  deliveryImageURI: URL?
  courier: Courier
  customer: User
}
```

```
/*
 * These are events the server sends us via a
 * RTC call or live web socket connection.
 * For OrderStatusChanged Event's we have a payload
 * dictionary so the server can send us more information.
 * For example it could contain the Courier object if a dispatch
 * has started. For cleaner typing each OrderStatus could have
 * its own "Changed" event, and instead of a dictionary it could
 * have its own typed Info object.
 */
CourierLocationUpdatedEvent {
  location: CLLocation
}
```

```
OrderStatusChangeEvent(s) {
  status: OrderStatus
  payloadDictionary: [String : String]
}
```

Open-ended Questions

1. Do we want to support only restaurants (how about other businesses)?
2. If ordering from nonrestaurant do we need to support item substitutions?
3. Any item can be customized or configured (i.e. Large or X-Large pizza, soda flavor).
4. Do we need to support "Standardized customizations or not?
5. Offline Mode? These apps in general don't make sense to support offline.
6. Multi-Order support? Can the user place multiple orders at the same time?
7. Will require a "Orders" screen, and a way of tracking multiple "Baskets"
8. How about item availability? What if an item in a basket is no longer available from the restaurant or merchant? Does the individual item get refunded, substituted, or order canceled?

Interesting Observations

1. Offline does not seem to be a thing for these apps, without connectivity you can't really place an order.
2. User can change the active address at any time. This means that a Basket can be for a restaurant at AddressA, but the active delivery address has now been changed to AddressB.
3. There are also seems to be a concept of different addresses. There is a delivery address, but there is also an "active" address that can be changed in the RestaurantList. It seems feasible that the active browsing address can change so a user can browse different restaurants in different locations, but the delivery address can be left to a different location entirely.
4. Address-level validation will be needed when placing an order.
5. How would we handle the concept of time and timezone? Have to make sure the restaurant is open and available for deliveries. The user's timezone doesn't matter as much as the merchant's does.

Technical Optimizations

- The app needs network connectivity but we can do our best to optimize/throttle network costs, optimize battery usage, and keep the UX responsive/smooth.
- We'll leverage pagination to keep the RestaurantList and Menu list lower latency calls.
- Cache images on client for better performance. Most images will be static anyway.
- Do what we can on client. For example the ShoppingCart/Basket creation is totally local and we don't need to communicate with the server. At the final "Checkout" the items in the basket can be validated on the server.
- We can make "baskets" idempotentable to ignore the UX and allow users to pick up where they left off.
- Get user location one time on app load.
- Only get real-time updates for user location on Order Status Screen.
- Let the server track and communicate the location of driver.

Comments on the Shopping Cart, Order Design

1. ShoppingCartBasket serves as an adapter and a facade to easily create a shopping cart. The idea is to make it easy to create a shopping cart, add items, and quickly display the total on the UI. It delegates the heavy lifting of totals calculation and currency localization to the Invoice object while providing a clean interface for simply adding basket items, getting formatted display totals for quick UI display (almost like a ViewModel), and getting the underlying invoice when its time to place an order.
2. Invoice object is more of a generic invoice calculating package. It handles the complexity of calculating all line item totals, applying line item discounts and taxes, and computing the global total and applying any global discounts and taxes. It also does the heavy lifting of formatting and localizing the calculation based locale (currency code), which can be pretty complicated. Leverages *NSNumberFormatter*.
3. A order is created on the backend using a combination of (merchantID, userID, invoice). The invoice is crucial. Its equivalent of doing a `invoice.save()` on the backend and starting a order/transaction (orderID)

Data Model Design Feedback & Critique

I chose for the client to handle currency localization and invoice calculation.

Pros:

- Keeps the network load & latency down. I felt it might be expensive to make a round trip to the server to update invoice calculation every time we add items to the shopping cart.

Cons:

- Invoice calculation logic is very complex and becomes quite brittle especially when supporting different currency codes. Because the server is already localizing strings, its more scalable to have the server side handle invoice calculation and localize the calculation as well.
- Duplicate implementations on the client side.

I believe doing the invoicing and currency localization was the wrong design decision, turns our app into a invoicing SDK and introduces mPOS-like complex business logic. Should keep the logic and performant lists and infrastructure for the UX (image caching). Although the original intention was good its a bit over-engineered for the use case. Because the app pretty much completely relies on a internet/cell connection it seems reasonable to hit a endPoint everytime an item is added to our basket to get the latest shoppingCart and it's totals. Invoicing almost becomes a purely backend implementation and clients can rely on one source of truth for the invoice calculation and currency localization logic.

The reason we calculated invoices on the client side at Paypal is because we are supporting mPOS apps. Merchants on a point of sale need to add and update items extremely quickly, they expect to see the totals updated right away. Furthermore mPOS systems need to be able to work offline. If connectivity goes down business should still continue so mPos need to support offline transactions. As a result invoicing being calculated client side makes complete sense for mPOS apps. It doesn't make much sense for a DoorDash which is really a "online" marketplace.